

1.11. Les variables dans leur propre fichier

x, y, MOT et le Mot déménagent dans « variables.h » ; main() n'a plus que les gestes.

1. Ce qu'il faut comprendre

Les quatre lignes du haut de la 1.10 sont maintenant dans un **second fichier**, `variables.h` : c'est l'onglet à côté de `principal.cpp`.

`#include "variables.h"` **verse ce fichier à cet endroit**, avant la compilation. Le programme est exactement le même que celui de la 1.10 ; il est seulement rangé en deux.

D'un côté ce qu'on affiche et où, de l'autre ce qu'on en fait. Pour changer le mot ou sa place, on n'ouvre que `variables.h`.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1341 octets** de cartouche.

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

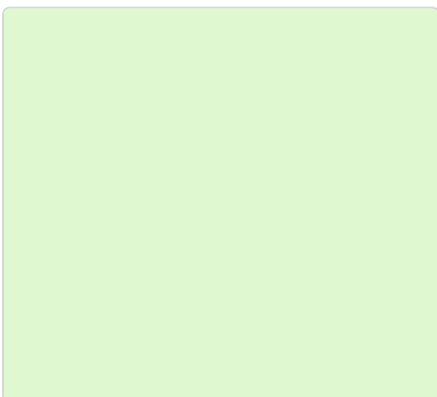
```
#include "variables.h"
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {  
    texte(mot_xy);  
    effacer(mot_xy);  
  
    while (true) image();  
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1

4. Ce qu'on doit voir

« BONJOUR » est écrit puis effacé en (x, y) : l'écran reste vide.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ x et y viennent de `variables.h` : 5 et 6
- ✓ les sept cases de `BONJOUR` sont vides
- ✓ et l'écran est allumé

Ensuite : **2. Effacer quand on appuie sur A** — niveau 1 sur 10. · `gameboy3` — écrire une cartouche Game Boy en C++